Vidyavardhini's College of Engineering &Technology
Department of Computer Engineering
Academic Year: 2026-27

Name of the Student: Mohammed Shahiduddin Shahalam
Class and Div and Roll No: TE-2/71
Experiment No.7
Aim: Implementation of a UDP Client–Server Application Using Socket Programming
Date of Performance: 18/08/2026
Date of Submission: 27/08/2026

Experiment No 7

AIM: Implementation of a UDP Client–Server Application Using Socket Programming

THEORY:

Socket Programming is a method of communication between two devices over a network using sockets. A socket is one endpoint of a two-way communication channel between two applications. Socket programming enables processes running on different computers to exchange data over a network.

The **User Datagram Protocol (UDP)** is a **connectionless transport layer protocol**. Unlike TCP, UDP does not establish a connection before transmitting data. It sends data in the form of independent packets called **datagrams**. UDP does not guarantee delivery, ordering, or error-free transmission of data. Because of its simple and connectionless nature, UDP provides faster communication with less overhead compared to TCP.

In a UDP Client–Server architecture, two programs participate in communication:

- **Server:** The server creates a socket and binds it to a specific IP address and port number. It waits for datagrams sent by clients, receives the data, processes the request, and sends an appropriate response back to the client.
- **Client:** The client creates a socket and sends a datagram to the server's IP address and port number. It does not establish a connection with the server. It waits for the server's response and receives the datagram.

Working of UDP Client–Server Communication

1. The server creates a UDP socket.
2. The server binds the socket to an IP address and port number.
3. The server waits for incoming datagrams from clients.
4. The client creates a UDP socket.
5. The client sends a datagram to the server's IP address and port number.
6. The server receives the datagram using recvfrom().
7. The server processes the received data.
8. The server sends a response to the client using sendto().
9. The client receives the response.
10. After communication is complete, the client and server close their sockets.

Note: Unlike TCP, UDP does not use listen(), accept(), or connect() for establishing a connection. The client and server communicate by directly sending and receiving datagrams.

Common Socket Functions

Function	Description
socket()	Creates a new socket.
bind()	Assigns an IP address and port number to the server socket.
sendto()	Sends a datagram to a specified destination address.
recvfrom()	Receives a datagram along with the sender's address.
close()	Closes the socket and terminates communication.

Important Difference Between TCP and UDP

TCP	UDP
Connection-oriented protocol	Connectionless protocol
Establishes a connection before communication	No connection establishment
Reliable data delivery	Does not guarantee delivery
Data is delivered in order	No guarantee of ordering
Uses listen() and accept() on server	Does not use listen() or accept()
Uses send() and recv()	Uses sendto() and recvfrom()
Higher overhead	Lower overhead
Comparatively slower	Comparatively faster

PROGRAM FOR UDP USING JAVASCRIPT(NODE JS):

Client Side Program:

```
const dgram = require("dgram");
const readline = require("readline");

const server = dgram.createSocket("udp4");
const PORT = 8080;

const rl = readline.createInterface({ input: process.stdin });

server.bind(PORT, () => {
   console.log(`Server is listening on port ${PORT}...`);
});

server.on("message", (msg, rinfo) => {
   console.log(`Client (${rinfo.address}:${rinfo.port}): ${msg}`);

   rl.question("Server: ", (reply) => {
      server.send(reply, rinfo.port, rinfo.address, () => {
         console.log("Reply sent to client.");
      });
   });
});
```

Server Side Program:

```
const dgram = require("dgram");
const readline = require("readline");

const client = dgram.createSocket("udp4");
const SERVER_PORT = 8080;
const SERVER_HOST = "127.0.0.1";

const rl = readline.createInterface({ input: process.stdin });

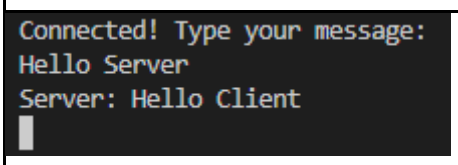
console.log("Connected! Type your message:");

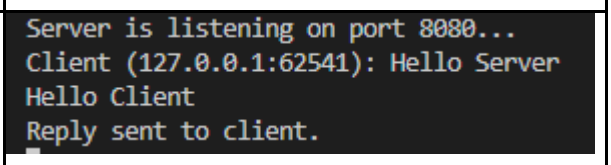
function askAndSend() {
   rl.question("Client: ", (message) => {
      client.send(message, SERVER_PORT, SERVER_HOST);
   });
}

client.on("message", (msg) => {
   console.log(`Server: ${msg}`);
   askAndSend();
});

askAndSend();
```

OUTPUT:





CONCLUSION: Thus, the UDP client-server application was successfully implemented using socket programming. The client and server communicated by sending and receiving datagrams without establishing a dedicated connection. This experiment helped in understanding connectionless communication, UDP sockets, datagram-based data transmission, and the differences between TCP and UDP communication.